

Universidad Rafael Landívar.
Facultad de Ingeniería.
Licenciatura en Ingeniería en Informática y Sistemas.
Estructura de Datos I
Docente: Ing. Juan Pablo Valiente

MANUAL TÉCNICO

“Generador de Imágenes por Capas”

Estudiantes: Velásquez González, Luis Manuel
Carné: 1502325

Quetzaltenango, 25 de mayo de 2026

1. Introducción

El presente Manual Técnico describe detalladamente la arquitectura, el diseño algorítmico y el funcionamiento interno de la aplicación **"Generador de Imágenes por Capas"**. Este sistema fue desarrollado nativamente en el lenguaje C++ , aplicando de manera estricta el paradigma de Programación Orientada a Objetos (POO) y cumpliendo con los lineamientos del curso de Estructura de Datos I. La premisa principal del desarrollo fue la implementación desde cero de todas las estructuras subyacentes, garantizando un control absoluto sobre el "Heap" de memoria mediante el uso de punteros y referencias cruzadas, omitiendo por completo el uso de librerías de colecciones estándar (STL) o frameworks externos para el almacenamiento de datos.

A nivel arquitectónico, el sistema resuelve el problema de la representación gráfica optimizada mediante la interconexión de diversas estructuras de datos dinámicas, tanto lineales como no lineales. El núcleo de almacenamiento de píxeles se delegó a un sistema de Matrices Dispersas Ortogonales. Esta decisión técnica es fundamental, ya que al no existir dimensiones fijas para los lienzos, el programa crea nodos exclusivamente en las coordenadas espaciales (X, Y) donde existe un valor de color hexadecimal. Las áreas vacías se ignoran a nivel de memoria, lo que representa un ahorro de recursos exponencial en comparación con una matriz tradicional.

Para orquestar estas matrices y el flujo de información, el sistema implementa múltiples estructuras complejas:

- **Árboles Binarios (ABB y AVL):** Se implementaron para garantizar tiempos de búsqueda logarítmicos $O(\log n)$. El catálogo de capas se estructura en un Árbol Binario de Búsqueda regular , mientras que la gestión de usuarios exige un árbol auto-balanceable para asegurar la eficiencia del registro y la autenticación.
- **Listas Circulares y Simples:** El catálogo maestro de imágenes reside en una Lista Circular Doblemente Enlazada, permitiendo una navegación continua. Para evitar la redundancia de datos, las relaciones jerárquicas de "qué capas componen una imagen" se administran mediante Listas Simples que guardan únicamente punteros de memoria hacia los nodos del árbol original de capas.

Además de la arquitectura de datos, este documento detalla el funcionamiento de los módulos complementarios críticos del sistema: el motor de Carga Masiva (que sanitiza y procesa archivos .cap, .im y .usr implementando bloques try-catch para tolerar fallos de sintaxis) , el motor de renderizado que exporta el arte final en código HTML mediante la superposición matemática de nodos , y finalmente, la integración con el motor gráfico Graphviz, el cual audita el estado interno de los punteros generando representaciones visuales exactas de las estructuras en tiempo de ejecución.

2. Arquitectura del Sistema

El sistema está dividido de manera modular, separando la lógica de las estructuras de datos, los controladores de lectura de archivos y la interfaz principal (Menú).

La lógica central se basa en la interconexión de estructuras: los catálogos principales (Árboles y Listas Circulares) almacenan los objetos reales en la memoria del "Heap", mientras que las estructuras secundarias (Listas simples dentro de Imágenes y Usuarios) utilizan punteros hacia dichos objetos para evitar la duplicación de datos y eficientar el uso de la memoria RAM.

3. Estructuras de Datos Implementadas

3.1. Matriz Dispersa Ortogonal (Capas)

La estructura más granular del sistema. Almacena los píxeles individuales de cada capa.

- **Nodos:** Cada nodo contiene coordenadas (x, y) y el color en formato Hexadecimal (std::string). Cada nodo posee 4 punteros: arriba, abajo, izquierda, derecha.
- **Cabeceras:** Se utilizan listas enlazadas para los índices de filas y columnas, facilitando la inserción dinámica sin un tamaño predefinido de lienzo.
- **Optimización:** Según los requerimientos, las áreas vacías y los colores blancos (#ffffff) no generan un nodo físico en la memoria, manteniendo la naturaleza "dispersa" y ahorrando recursos durante el renderizado.

3.2. Árbol Binario de Búsqueda de Capas

- **Estructura Global:** Organiza todas las capas cargadas al sistema utilizando el ID de la capa como clave de ordenamiento (los menores a la izquierda, mayores a la derecha).
- **Nodos:** Almacena un puntero hacia la estructura LayerMatrix (Matriz Dispersa).
- **Recorridos:** Implementa métodos recursivos de extracción limitada en Preorden, Inorden y Postorden, lo cual es vital para la generación de imágenes por profundidad de capas.

3.3. Árbol de Búsqueda (AVL) de Usuarios

- **Balanceo:** Almacena a los usuarios registrados garantizando un tiempo de búsqueda óptimo de $O(\log n)$.
- **Relaciones:** Cada nodo de usuario (identificado por su username en formato string) contiene un puntero hacia la cabeza de una lista simple de imágenes, la cual funge como su historial/galería personal.

3.4. Lista Circular Doblemente Enlazada (Catálogo de Imágenes)

- **Comportamiento:** Almacena de forma global todas las imágenes válidas del sistema. El puntero siguiente del último nodo apunta a la cabeza, y el anterior de la cabeza apunta al último nodo.
- **Estructuras Anidadas:** Cada nodo (Imagen) posee una lista simple de capas. Esta lista simple almacena de forma ordenada la jerarquía en la que se deben superponer las capas, guardando únicamente *punteros* hacia los nodos originales alojados en el Árbol Binario de Capas.

4. Flujos de Procesamiento Críticos

4.1. Carga Masiva y Sanitización de Datos

El módulo CargaMasiva.h utiliza manipulación de cadenas de texto en C++ para interpretar los archivos .cap, .im y .usr.

- **Manejo de Errores (Excepciones):** Se implementaron bloques try-catch para blindar la ejecución ante errores de sintaxis en los archivos. Si `std::stoi()` falla al intentar convertir una cadena alfanumérica, el sistema captura la excepción (`std::invalid_argument`), reporta el error y continúa con la siguiente instrucción, impidiendo un cierre abrupto (Crash) del sistema.
- **Manejo de Rutas:** Se emplea `std::getline()` y `std::cin.ignore()` para permitir la lectura absoluta de rutas del sistema operativo que contengan espacios en blanco en sus nombres de carpetas.

4.2. Renderizado de Imágenes HTML (Superposición)

El módulo GeneradorImágenes.h extrae las coordenadas de las matrices dispersas seleccionadas.

1. Se determina el tamaño máximo del lienzo (Max X y Max Y) iterando sobre las cabeceras de las matrices involucradas.
2. Se procesan las matrices en el orden de la lista simple (de la primera insertada a la última). Al pintar en un canvas HTML/CSS usando la etiqueta `<div>` o `<table>`, los colores de las últimas capas sobreesciben a las primeras, generando el efecto de superposición.
3. Si el algoritmo detecta una imagen sin ninguna capa asociada en su lista simple, automáticamente genera la salida HTML renderizando un único píxel de color negro, cumpliendo la regla de excepción de imágenes vacías.

5. Reportes del Estado de Memoria (Graphviz)

La auditoría de las estructuras se realiza generando scripts en formato .dot que son procesados por el motor de renderizado de Graphviz a través del comando system().

- **Resolución de Conflictos Estructurales:** Para la correcta visualización gráfica de la Lista Circular Doblemente Enlazada, se detectó el bug conocido de Graphviz *"flat edge between adjacent nodes"*. Esto se resolvió descartando las formas record tradicionales y construyendo la representación del nodo mediante etiquetas pseudo-HTML (<TABLE>), lo que permite conexiones cíclicas perfectas en la misma jerarquía (rank=same).
- **Sanitización de Símbolos:** Las etiquetas inyectadas a Graphviz pasan por un proceso de sanitización para eliminar caracteres acentuados o especiales que rompen el formato de análisis XML interno de la herramienta dot.

6. Especificaciones de Compilación y Ejecución

Para que otro desarrollador pueda modificar o recompilar el proyecto, debe asegurar el siguiente entorno:

- **Lenguaje:** C++ (Estándar C++11 o superior recomendado).
- **Compilador:** GCC/G++ (vía MinGW en entornos Windows) o equivalente en Linux.
- **Librerías:** Únicamente Standard Template Library para flujos I/O y strings (<iostream>, <string>, <fstream>, <sstream>, <vector>, <exception>). No se emplean librerías <list>, <map> ni frameworks de terceros.
- **Dependencia Externa de Ejecución:** El entorno del sistema operativo debe contar con los binarios de **Graphviz** agregados en la variable PATH del sistema para que el comando dot -Tpng no devuelva error de comando desconocido.